CS201 Lab 4

Data Structures and Algorithms

Shreya Agarwal Rachit Nimavat

Spring 2026

This lab builds directly upon your `IntVector` library from Labs 2 and 3. You will need your `vector.h`, `vector.c`, and `Makefile`. If your implementation of `IntVector` was buggy, you may start fresh by downloading a copy from the CS201 labs Github.

Part 1: Stock Price History

You are given daily stock prices of a company in an array. Given a list of daily stock prices, calculate the "span" for each day. The span is the maximum number of consecutive days (backwards from the current day) that the price has been less than or equal to that day's price. Write a $O(n^2)$ solution first, and then optimize it to $O(n)$ time using a well-known[1] data structure.

Example: Input: `[100, 80, 50, 60, 70, 40, 90]` Output: `[1, 1, 1, 2, 3, 1, 6]`

Part 2: Linked List based IntVector

Our `IntVector` supports `append()`, `search()` and `pop_index()` operations with a growing-and-shrinking array as a backend. Update your `IntVector` library to implement all these operations using a linked list as a backend. With this, verify that your code for part 2 of lab-3 still works.

Questions to Ponder Over this Week

1. Your $O(n)$ solution for part 1 likely used a nested loop in some form. Usually, nested loops indicate a time complexity of $O(n^2)$. Explain formally how your time complexity is still $O(n)$ despite the nested loop. The analysis that you do to prove this is called **Amortized Analysis**, where we track a *potential function* (like "energy" stored in the data structure) over the entire execution of the algorithm.
2. In Lab 2, when our `IntVector` ran out of space, we doubled the capacity ($2 \times N$). This felt wasteful. Rachit suggests: "Why not just add space for 1000 more elements ($N + 1000$)? That saves memory!" Prove that his strategy is a disaster using Amortized Analysis. The Scenario: You perform M appends. The Cost: In the doubling strategy, the amortized cost per append is $O(1)$. In the additive strategy, what is the total complexity for M appends?

In this lab, we swapped out the backend of `IntVector` from a dynamic array to a linked list.

[1] Use a (monotonic) stack.

3. In the Array version, accessing the k-th element of `IntVector` took $O(1)$ time. What is the time complexity of accessing the k-th element in the Linked List version? Does this change the overall time complexity of your Josephus solution from Lab 3? If so, from what to what?

4. Differentiate between the time complexity of `append()`, `search()` and `pop_index()` between the two versions of `IntVector`. When should you prefer one over the other?

5. In the Array version, storing `1000` integers took roughly `1000 * 4 = 4000` bytes. In the Linked List version, you need a Node struct for every integer. Each Node contains an integer (4 bytes) and a pointer to the next Node (8 bytes on a 64-bit system). So, each Node takes `4 + 8 = 12` bytes and for `1000` integers, you need `1000 * 12 = 12000` bytes. Is the flexibility of the linked list worth this penalty? What do you gain from using a linked list over an array?